Binary Search: O(LogN):

```
public static int binarySearch(int[] numbers,int low, int high,int key) {
    while (high >= low) {
        int mid = (low + high) / 2;
        if(numbers[mid] < key){
            low = mid + 1;
        }
        else if (numbers[mid] > key) {
            high = mid - 1;
        }
        else {
            return mid;
        }
    }
    return -1;
    }
```

Linear Search: O(N):

```
LinearSearch(numbers, numbersSize, key) {
    i = 0

    for (i = 0; i < numbersSize; ++i) {
        if (numbers[i] == key) {
            return i
        }
    }

    return -1 // not found
}
```

List ADT's:
Arrays: Append/Prepend O(N), InsertAfter, Remove, Contains, IsEmpty
Singly Linked List: SLL Class Members: Nodes (Head, Tail, Temp), Node Class (this.val, this.next)

```
public void appendNode (AnyType data) {
    Node<AnyType> node = new Node<>(data);
    if (head == null) {
        head = node;
        tail = node;
    }
    else {
        tail.next = node;
        tail = node;
    }
}
```

```
public void preppendNode (AnyType data) {
    Node<AnyType> node = new Node<>(data);
    if (head == null) {
        head = node;
        tail = node;
    }
    else {
        temp = head;
        head = node;
        head.next = temp;
    }
}
```

```
public class SinglyLinkedList<AnyType> {
    private Node<AnyType> head;
    private Node<AnyType> tail;
    private Node<AnyType> temp;

    public SinglyLinkedList() {
        this.head = null;
        this.tail = null;
        this.temp = null;
    }
    private static class Node<AnyType>{
        public Node(AnyType val) {
            this.val = val;
            this.next = null;
        }
        public AnyType val;
        public Node<AnyType> next;
    }
```

Insert: O(N)
Remove: O(N)
Search: O(N)

Doubly Linked List: SLL Class Members: Nodes (Head, Tail, Temp), Node Class (this.val, this.next, this.prev)
Insert: O(N) Remove: O(N) Search: O(N), supports reverse traversal.
Stack (LIFO): Push O(1), Pop O(1), Search O(N) | Push -> stack[tail++] = item | Pop -> item = stack[—tail]
Queue (FIFO): Enqueue O(1), Dequeue O(1), Search O(N) | Enqueue -> queue[tail++] = item | Dequeue -> item = queue[head++]
Hash Map: Key: Value, collisions occur when multiple Vals have the same key. Linear Probing, Quadratic Probing, Double Hash

```
public class SeparateChainHashTable<AnyType> {
    private static final int LIST_SIZE = 10;
    private List<AnyType> [] theLists;
    public SeparateChainHashTable() {
        this(LIST_SIZE);
    }
    @SuppressWarnings("unchecked")
    public  SeparateChainHashTable( int size) {
        theLists = new LinkedList[size];
        for (int i = 0; i < theLists.length; ++i) {
            theLists[i] = new LinkedList<>();
        }
    }
    private int myHash(AnyType x) {
        int hashVal = x.hashCode() % 10;
        return hashVal;
    }
    public boolean contains(AnyType x) {
        List<AnyType> whichList = theLists[myHash(x)];
        return whichList.contains(x);
    }
    public void insert(AnyType x) {
        List<AnyType> myList = theLists[myHash(x)];
        if(!myList.contains(x)) {
            myList.add(x);
        }
    }
```

```
public class LinearProbingHashTable<AnyType> {
    private static final int LIST_SIZE = 10;
    private AnyType[] theList;
    public LinearProbingHashTable() {
        this(LIST_SIZE);
    }
    @SuppressWarnings("unchecked")
    public  LinearProbingHashTable( int size) {
        theList = (AnyType[]) new Object[size];
    }
    private int myHash(AnyType x) {
        int hashVal = x.hashCode() % 10;
        return hashVal;
    }
    public void insert(AnyType x) {
        int bucketIndex = myHash(x);
        for(int i = 0; i < LIST_SIZE; ++i) {
            int probe = (bucketIndex + i) % LIST_SIZE;
            if(theList[probe] == null) {
                theList[probe] = x;
                return;
            }
        }
    }
```

```
public class QuadraticProbingHashTable<AnyType> {
    private static final int LIST_SIZE = 10;
    private AnyType[] theList;
    public QuadraticProbingHashTable() {
        this(LIST_SIZE);
    }
    @SuppressWarnings("unchecked")
    public  QuadraticProbingHashTable( int size) {
        theList = (AnyType[]) new Object[size];
    }
    private int myHash(AnyType x) {
        int hashVal = x.hashCode() % 10;
        return hashVal;
    }
    public void insert(AnyType x) {
        int bucketIndex = myHash(x);
        for(int i = 0; i < LIST_SIZE; ++i) {
            int probe = (bucketIndex + (i*i)) % LIST_SIZE;
            if(theList[probe] == null) {
                theList[probe] = x;
                return;
            }
        }
    }
```

```
public class DoubleHashTable<AnyType> {
    private static final int LIST_SIZE = 10;
    private Integer[] theList;
    public DoubleHashTable() {
        this(LIST_SIZE);
    }
    public  DoubleHashTable( int size) {
        theList = (Integer[]) new Integer[size];
    }
    private int myHash(int x) {
        int hashVal = x  % 10;
        return hashVal;
    }
    private int myDoubleHash(int x) {
        int hashVal = 7 - (x % 7);
        return hashVal;
    }
    public void insert(int x) {
        int bucketIndex = myHash(x);
        int i = 0;
        while (i < LIST_SIZE) {
            int probe = (bucketIndex + i * myDoubleHash(x)) % LIST_S
            if(theList[probe] == null) {
                theList[probe] = x;
                return;
            }
        i += 1;
        }
    }
```

Chaining Collisions, Insert O(N) best case O(1) | Remove O(N) best case O(1) | Search O(N)
Linear Probing, Insert O(N) average case O(1) | Remove O(N) average case O(1) | Search O(N) average case O(N) (contiguous clustering)
Quadratic Probing | Insert O(N) - more overhead and prone to clustering (secondary clustering, keys follow the same probe sequence)
Double Hashing | Insert O(N) - more overhead more complex hash function, but way less prone to clustering
Quicksort: O(NLogN) on average, worst case O(N^). Repeatedly partition array in to low and high parts around a pivot value, if the low value, mid value, and high value are unsorted, swap them In the array and hide the pivot value right before the high value.
Mergesort: O(NLogN) on average, worst case O(N^). Identify low and high point, calculate mid, recursively call merge sort on left array, identify low medium and high until we have a singular array which is already sorted, then we call on the right array and call our merge method to sort smallest components, repeat on right side of array
Heapsort O(NLogN): create a max heap out can array, repeatedly swap root with last element in array, and percolate the swapped value down, repeat until your array is sorted

Build max heap:
To build min heap switch sign in first picture

```java
@SuppressWarnings("unchecked")
public BinaryHeap(int capacity) {
    currentSize = 0;
    array = (AnyType[]) new Comparable[capacity +1];
}

@SuppressWarnings("unchecked")
public BinaryHeap(AnyType[] items) {
    currentSize = items.length;
    array = (AnyType[]) new Comparable[(currentSize

    int i = 1;
    for(AnyType item:items )
        array[i++] = item;
    printHeap();
    buildHeap();
}

private void buildHeap() {
    for (int i = currentSize/2; i > 0; i--) {
        percolateDown(i);
}
```

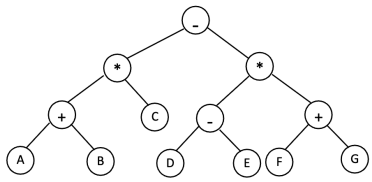
```java
}

private void percolateDown(int hole) {
    int child;
    AnyType tmp = array[hole];

    for(; hole * 2 <= currentSize; hole = child ) {
        child = hole * 2;
        if(child != currentSize && array[child + 1].compareTo(array[child]) > 0 ) {
            child++;
        }
        if(array[child].compareTo(tmp) > 0 ) {
            array[hole] = array[child];
        }
        else {
            break;
        }
    }
    array[hole] = tmp;
    printHeap();
}
```

```java
public AnyType findMax() {
    //return the first element in the max heap
    return array[1];
}

public void deleteMax() {
    //return the first element in the max heap, which is always the max
    AnyType max = findMax();
    //show the current size for help
    System.out.println("current size is: " + currentSize);
    //put the last array element at the front of the list
    array[1] = array[currentSize];
    //put the last element of the list as the max
    array[currentSize] = max;
    //percolate the new root node down as it is not the max always and decrement cur
    //keep swapping them in and out of place
    --currentSize;
    percolateDown(hole:1);
}
```



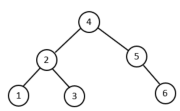
Prefix: math operations precedence parenthesis

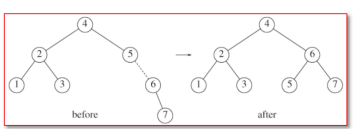
Postfix AB+C*DE-FG+*- (letters then signs)
Prefix -*+ABC*-DE+FG (signs then letters)
Infix (normal rotation)

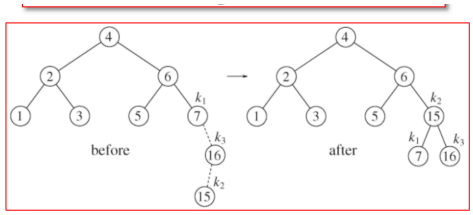
Minimum spanning trees = Krystal or Prim's algorithm (Prim takes successive best course of action)
AVL Trees:

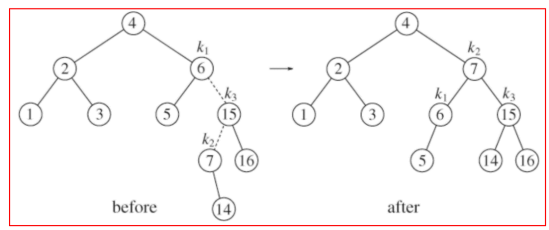
Q4. Show the result of inserting 7, 16, 15, 14 items into the following binary AVL tree. Draw a tree after inserting each individual item (make sure they are all AVL trees):

This is an example from the DSAA book









All Pairs Shortest Path and Dijkstra's pseudocode

```java
private static int[][] allPairs(int[][] adjMatrix) {

    int[][] shortPath = new  int [adjMatrix.length][adjMatrix.length];
    int[][] path = new  int [adjMatrix.length][adjMatrix.length];


    for(int i = 0; i < adjMatrix.length; ++i) {
        for(int j = 0; j < adjMatrix.length; ++j) {
            shortPath[i][j] = adjMatrix[i][j];
            path[i][j] = NOT_A_VERTEX;
        }
    }
    for(int k = 0; k < adjMatrix.length; ++k) {
        for(int l = 0; l < adjMatrix.length; ++l) {
            for(int m = 0; m < adjMatrix.length; ++m) {
                if(shortPath[l][k] + shortPath[k][m] < shortPath[l][m])
                    shortPath[l][m] = shortPath[l][k] + shortPath[k][m];
                    path[l][m] = k;
                }
            }
        }

    return shortPath;
}
```

Directed acyclic graphs do not have cycles,
A cycle is formed if a vertex is visited more than once
Dijkstra's algorithm can work with cyclic graphs

14. Dijkstras's Algorithm, worst with regular list O(V^2), priority queue O((V+E)logV)
Floyd-Warshall All-pairs shortest paths Time complexity O(V^3) SpaceO(V^2)

```
void dijkstra( Vertex s )
{
    for each Vertex v
    {
        v.dist = INFINITY;
        v.known = false;
    }

    s.dist = 0;

    while( there is an unknown distance vertex )
    {
        Vertex v = smallest unknown distance vertex;

        v.known = true;

        for each Vertex w adjacent to v
            if( !w.known )
            {
                DistType cvw = cost of edge from v to w;

                if( v.dist + cvw < w.dist )
                {
                    // Update w
                    decrease( w.dist to v.dist + cvw );
                    w.path = v;
                }
            }
    }
}
```

Topological sort: must be a directed acyclic graph,
Start with nodes that don't have any in degrees
And remove them from the graph, when you remove
The nodes, decrement the in degrees of the neighbor
Nodes, once the in degrees reaches 0, add it to the
List Of nodes